

# Computer Architecture

## Assignment 2

Deadline: Sunday, June 28, 2026

Mayer Goldberg

2026-06-06

### Contents

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>Guidelines</b>                          | <b>1</b> |
| <b>2</b> | <b>Computing logarithms</b>                | <b>1</b> |
| <b>3</b> | <b>The algorithm you need to implement</b> | <b>3</b> |
| <b>4</b> | <b>When to terminate</b>                   | <b>3</b> |
| <b>5</b> | <b>Submission</b>                          | <b>3</b> |

## 1 Guidelines

You are expected to conduct yourself honestly, sincerely, and honorably at all times. If you find you need help, the teaching staff is available and willing to assist you. I (Mayer) have solved this assignment in its entirety before handing out the assignment, I know where the difficulties lay, and I am happy to help you if you are stuck.

You may work on your assignment either alone or with a single partner. If you encounter difficulties, you may consult with the teaching staff.

The use of AI, classmates who are not your partners in for this assignment, friends, relatives, qualified physicians, *mechanical Turks*, electro-optic Mongols, illegal aliens, aliens from other galaxies, and various internet resources are *Extras*.

You are free to use *Extras* as a reference, to explain assembly instructions, to explain external functions from *clib* with which you might wish to link, or to debug. You may not use *Extras* to write code for this assignment. If you use *Extras* to write code, this shall constitute cheating, and your code shall be used as evidence in a disciplinary hearing (ועדת משמעת) held against you. The lightest penalty you are likely to receive for cheating is a failing grade in the course, though most often, additional penalties are added. Repeat offenses carry significantly higher penalties, including expulsion (הפסקת לימודים).

## 2 Computing logarithms

For this assignment, you are to write the standalone program `disc-log`. The program takes 3 floating-point numbers  $a, b, \varepsilon$ , on the command-line, and prints out  $\log_a b$  within the tolerance  $\varepsilon$ .

The algorithm will be presented here, and you must use this algorithm, rather than using the built-in x87 instruction for computing logarithms in base 2.

Below are some examples to demonstrate the behavior of the program:

```
gmayer@alonzo> make disc-log
nasm -g -f elf64 -l disc-log.lst disc-log.asm
gcc -g -m64 -no-pie -o disc-log disc-log.o
rm disc-log.o
gmayer@alonzo> ./disc-log shalom rav shuvech tsiporah nechmedet
Usage:
    <program> <base> <number> <epsilon>
gmayer@alonzo> ./disc-log 1 2
Usage:
    <program> <base> <number> <epsilon>
gmayer@alonzo> ./disc-log 2 3 4 5
Usage:
    <program> <base> <number> <epsilon>
gmayer@alonzo> ./disc-log 10 2 1e-17
log_{10}(2) = 0.301029995663981195
gmayer@alonzo> ./disc-log 10 3 1e-17
log_{10}(3) = 0.477121254719662437
gmayer@alonzo> ./disc-log 2 8 1e-17
log_{2}(8) = 3
gmayer@alonzo> ./disc-log 8 4 1e-17
log_{8}(4) = 0.666666666666666667
gmayer@alonzo> ./disc-log 10 7 0.1
log_{10}(7) = 0.846153846153846154
gmayer@alonzo> ./disc-log 10 7 0.01
log_{10}(7) = 0.845070422535211268
gmayer@alonzo> ./disc-log 10 7 0.001
log_{10}(7) = 0.845098039215686275
gmayer@alonzo> ./disc-log 10 7 0.0001
log_{10}(7) = 0.845098039215686275
gmayer@alonzo> ./disc-log 10 7 0.00001
log_{10}(7) = 0.845098039215686275
gmayer@alonzo> ./disc-log 10 7 1e-5
log_{10}(7) = 0.845098039215686275
gmayer@alonzo> ./disc-log 10 7 1e-10
log_{10}(7) = 0.845098040014256831
gmayer@alonzo> ./disc-log 10 7 1e-15
log_{10}(7) = 0.845098040014256831
gmayer@alonzo> ./disc-log 10 7 1e-17
log_{10}(7) = 0.845098040014256831
```

- If you call the program with insufficient or incorrect arguments, you will get a generic error message that clarifies the output of the program. As with all error messages under linux, it should be printed to `stderr`, and not to `stdout`. ☞ This is particularly important, because if you send error messages to `stdout` it will cause the grading-program to fail of your

assignment, and you shall lose points.

Your assignment shall be graded under Ubuntu Linux. You would do wisely to test your code at the linux labs on campus, just to make sure there are no last-minute surprises and incompatibilities.

### 3 The algorithm you need to implement

The algorithm is based on 3 simple properties of logarithms. For all  $a, b > 0$ , we have:

$$\log_a b \cdot \log_b a = 1 \quad (1)$$

$$\log_a a = 1 \quad (2)$$

$$\log_a b = 1 + \log_a \left( \frac{b}{a} \right) \quad (3)$$

So here is your algorithm:

- ▶ If  $a > b$ , return  $\frac{1}{\log_b a}$
- ▶ Otherwise, if  $\frac{b}{a} < \varepsilon$ , return 1
- ▶ Otherwise, return  $1 + \log_a \left( \frac{b}{a} \right)$

If you think hard about this algorithm, you should see that it has the same structure as *Euclid's Algorithm* for computing the *greatest common-divisor* (gcd). It would be fun and instructive to think why the two algorithms are essentially the same.

### 4 When to terminate

### 5 Submission

- ▶ Please submit a compressed folder by the name `hw2-student-ids.zip`. For example:
  - ▶ If you are submitting alone, and your Israeli ID number is 123456789
    - ▶ The name of your zip file should be `hw2-123456789.zip`
    - ▶ The name of the corresponding directory should be `hw2-123456789`
  - ▶ If you are submitting with a partner, your Israeli ID number is 123456789, and the ID of your partner is 987654321
    - ▶ The name of your zip file should be `hw2-123456789+987654321.zip`
    - ▶ The name of the corresponding directory should be `hw2-123456789+987654321`
- ▶ Unpacking the zip file should create the following directory structure:
  - ▶ `hw2-student-ids`
    - ▶ A file `readme.text`. This file should be a plain text file containing the following information:
      - ▶ For each student submitting, please include:
        - ▶ Full name

- ▶ Israeli ID
- ▶ Email address
- ▶ A brief paragraph asserting that
  - ▶ This work is your own
  - ▶ You have not used AI to write the code
  - ▶ You have not solicited and/or obtained help writing the code other than from the course staff and possibly each other
- ▶ A file `disc-log.asm` containing your work
- ▶ Please follow the instructions carefully, and do spend the extra time necessary to test, re-test, and make sure your code complies with the requirements, including the submission requirements. Your code shall be graded via an automatic, grading script. The graders shall not modify your code or allow you to resubmit if your code fails to comply. It is therefore essential to you follow the instructions carefully, to the letter, so as to avoid losing points unnecessarily.

The TAs shall post information about how to submit your assignment.